Mini-VECTO: quick start

TRA545 Project #4. Diesel, HVO100, H2-ICE, FCEV and BEV consumption of a timber truck, an urban bus and a coach on 100 km missions.

1. What you need

- A computer with **Python 3.9 or newer** (macOS, Windows or Linux). Download: python.org/downloads. On Windows tick "Add Python to PATH" in the installer.
- Three Python packages: `numpy`, `matplotlib`, `openpyxl`. The launcher installs them (internet needed the first time only).
- Excel (or LibreOffice) to open the results, and any browser for `index.html`. No internet is needed to run the model.

2. Run the model

1. Unzip the folder anywhere.
2. **Easy way:** double click `RUN_MODEL_mac.command` (macOS: the first time, right click > Open > Open) or `RUN_MODEL_windows.bat` (Windows).
3. **Terminal way:** open a terminal in the folder and type:

```
python3 -m pip install -r requirements.txt     (first time only; on Windows use "py"
instead of "python3")
python3 run_all.py
```

4. Wait about one minute. The terminal prints the five steps and then a table with the mean consumption per 100 km.

If only vehicle or powertrain data changed and the routes did not, `python3 run_all.py --keep-cycles` skips the route step. Use the full run after changing mass, drag, rolling resistance or engine power, because they change the speed profile.

3. Where the results are

File	Content
`results/consumption_Diesel.xlsx`, `_HVO100`, `_H2-ICE`, `_FCEV`, `_BEV`	One workbook per powertrain. **Summary**: mean consumption per mission and for the timber round trip. One sheet per mission with one row every 10 m. **Notes**: parameters and sources.
`index.html`	Mission explorer (speed, altitude, grade, road surface). Double click to open.
`consumption_*.csv`	The same step data as plain CSV, for emission factors or other tools.
`fig_*.png`	Mission profiles and engine operating points.

4. Where to change the parameters

Everything is in **`vehicles.json`**. Open it with a text editor (TextEdit, Notepad, VS Code), change a number, save, run again. Keep the commas and quotes as they are.

What	Section in vehicles.json	Keys

Vehicle mass, drag, rolling resistance	`missions` > mission name	`mass_kg`, `rot_mass_kg`, `CdA_m2`, `cr_asphalt`, `cr_gravel`
Engine size and power, speed limits	`vehicles` > vehicle > `engine`	`rated_power_kW`, `displacement_l`, `n_idle_rpm`, `n_max_rpm`
Axle ratio, tyre radius, auxiliaries (engine cases)	`vehicles` > vehicle	`axle_ratio`, `eta_axle`, `tyre_dynamic_radius_m`, `aux_mech_kW`
Gear ratios and gearbox efficiency	`gearboxes`	`ratios`, `eta_indirect`, `eta_direct`
Fuel properties, pilot share of the hydrogen engine	`fuels`, `hpdi`	`LHV_MJ_per_kg`, `density_kg_per_l`, `pilot_share_full_load`
Electric motor power, gears, electric auxiliaries	`electric` > `vehicles` > vehicle	`motor_peak_kW`, `n_gears`, `v_top_kmh`, `aux_el_kW`
BEV battery	`electric` > `vehicles` > vehicle > `bev`	`battery_kWh`, `soc_start`, `C_charge`, `extra_mass_kg`
Fuel cell power and buffer battery	`electric` > `vehicles` > vehicle > `fcev`	`fc_net_kW`, `battery_kWh`, `extra_mass_kg`
Fuel cell efficiency curve, minimum load, energy management	`electric` > `fuel_cell`	`eta_sys_BOL`, `eta_sys_EOL`, `min_load_frac`, `ems`
Motor losses, regenerative braking limits	`electric` > `edrive`	`machine_peak_efficiency`, `regen_adhesion_use`, `regen_v_cut_kmh`
Battery loss, charger efficiency	`electric` > `battery`	`loss_at_1C`, `charger_efficiency`
Routes: length, speed limits, stops, hills	not in the json: `missions.py`, section "MISSIONS"	route segments

Three things to know. (1) The engine map scales with `displacement_l`; `rated_power_kW` limits the speed profile. To make an engine bigger or smaller change both in proportion (the reference is 397 kW for 12.8 L). (2) The mission mass is shared by all powertrains. To make only the BEV or the FCEV heavier, use `extra_mass_kg`. (3) Vehicles: `timber_truck`, `urban_bus`, `coach`. Missions: `timber_empty`, `timber_loaded`, `urban_bus`, `coach`.

5. Five-minute live demo

1. Open `index.html`: show the loaded timber mission (speed, hills, gravel sections).
2. Open `vehicles.json`: show that all vehicle data sit in one file.
3. Change one number, for example the loaded truck `mass_kg` from 74000 to 64000. Save.
4. Run the model (double click the launcher). The table appears after about one minute.
5. Open `results/consumption_Diesel.xlsx`, sheet Summary, and compare with the previous value (67.1 l/100 km at 74 t). Then show one mission sheet: fuel every 10 m.
6. Set the value back and run again.

Run the model once before the meeting, so the packages are already installed and the result files exist even without internet.

6. If something goes wrong

Message	Fix
"python3: command not found" / "py is not recognized"	Install Python from python.org (Windows: tick "Add Python to PATH"), then reopen the terminal.
"Missing Python packages"	`python3 -m pip install -r requirements.txt`
"Stopped: ... failed" after editing	Usually a missing comma or quote in `vehicles.json`. Undo the last edit.
"Permission denied" when writing an .xlsx	Close the workbook in Excel and run again.
macOS refuses to open the launcher	Right click the file > Open > Open. Or use the terminal way.
"Steps with demand above the powertrain limit" is not 0 in a Summary	The powertrain is too weak for the cycle (for example motor or fuel cell too small). Increase its power or accept that the result is optimistic.